

# SSD Architecture and Performance Benefits

## Abstract

Solid-State Drives (SSDs) have revolutionized storage by eliminating moving parts and leveraging high-speed flash memory. This paper presents an overview of SSD architecture and examines the performance benefits of SSDs over traditional Hard Disk Drives (HDDs) and across different SSD interface types (SATA, NVMe, PCIe, M.2, U.2). We detail the internal architecture of SSDs – including flash memory arrays, controllers, and caches – and explain how design features (parallelism, advanced controllers, and optimized interfaces like NVMe) yield dramatic improvements in input/output operations per second (IOPS), throughput, and latency. Comparisons show that SSDs deliver orders-of-magnitude higher IOPS and much lower access latencies than HDDs <sup>1</sup> <sup>2</sup>. Furthermore, modern NVMe SSDs connected via PCI Express outperform SATA-based SSDs by substantial margins due to a streamlined protocol and greater bandwidth <sup>3</sup> <sup>4</sup>. Diagrams and quantitative data are provided to illustrate these performance gains. Overall, SSDs' architectural advances translate into significantly faster data access, making them the dominant choice for performance-critical storage in both client and enterprise environments.

## Introduction

In recent years, SSDs have become the primary storage medium for many applications, replacing or augmenting HDDs. Unlike HDDs, which store data on spinning magnetic platters and require mechanical head movements, an SSD stores data in semiconductor memory (typically NAND flash) and has no moving parts <sup>5</sup>. This fundamental difference in architecture leads to **faster data access speeds** and **lower latency**. HDDs typically exhibit average seek + rotational latencies on the order of 10–20 milliseconds, whereas SSD access latencies are often in the range of microseconds to a few tenths of a millisecond <sup>6</sup> <sup>7</sup>. In practical terms, an enterprise 15,000 RPM HDD might handle roughly 150–180 random IOPS, while even consumer SATA SSDs can achieve tens of thousands of IOPS, and NVMe SSDs can exceed a million IOPS <sup>8</sup> <sup>3</sup>. SSDs also provide far greater sequential throughput, often saturating the limits of their interface (e.g. ~550 MB/s on SATA, versus ~100–250 MB/s for a 7,200 RPM HDD) <sup>9</sup> <sup>10</sup>.

Beyond speed, SSDs offer additional benefits: they draw less power, run cooler, and are shock-resistant due to the lack of mechanical parts <sup>5</sup> <sup>11</sup>. These advantages come at a higher cost per gigabyte and the intrinsic limitation that flash memory can endure only a finite number of write/erase cycles. Nevertheless, for performance-critical applications and everyday computing alike, the **performance gains** of SSDs overwhelmingly justify their adoption. This paper explores how SSD architecture enables these gains. We first outline the internal design of SSDs, then compare the performance characteristics of various interfaces (SATA vs. NVMe) and media types, highlighting why SSDs so vastly outperform HDDs.

## SSD Architecture Overview

At a high level, an SSD consists of three primary components: the **flash memory** (NAND flash chips for data storage), the **SSD controller** (an embedded processor that manages data placement and retrieval), and

typically a **DRAM cache** or buffer <sup>12</sup> <sup>13</sup> . These components are assembled on a printed circuit board along with a power management circuit. **Figure 1** illustrates a simplified block diagram of a typical SSD controller architecture, including the host interface logic, controller/processor, DRAM buffer, and multiple flash memory channels:

<sup>14</sup> <sup>15</sup> *Figure 1: Simplified block diagram of an SSD controller, showing the host interface (e.g., SATA or PCIe), controller SoC with embedded CPU cores and firmware, DRAM cache, and an array of NAND flash memory chips on multiple channels. The SSD controller's firmware implements functions like wear leveling, error correction, garbage collection, and mapping of logical blocks to physical flash locations (Flash Translation Layer).* <sup>16</sup> <sup>17</sup>

In an SSD, the **controller chip is the heart of the device**, bridging the host interface to the flash media <sup>18</sup> . The controller runs firmware that handles flash management tasks: it translates host read/write commands into operations on the flash chips and maintains a mapping table (FTL) to track where each logical block address resides in the physical NAND flash <sup>19</sup> . It also performs **wear leveling** (spreading writes evenly across cells to extend lifespan) and **garbage collection** (reclaiming space by erasing blocks with stale data) in the background <sup>20</sup> <sup>17</sup> . Modern controllers typically include error-correcting code (ECC) engines to detect and fix bit errors, and often encryption engines for data security <sup>21</sup> <sup>22</sup> . High-performance SSD controllers integrate multiple ARM/RISC cores and specialized hardware accelerators to manage these tasks in parallel without bottlenecking the I/O path <sup>23</sup> <sup>24</sup> .

**Flash memory architecture:** NAND flash is organized into arrays of memory cells, pages, and blocks. Reads and writes occur in page-sized chunks (e.g., 4 KB or 16 KB), while erases happen at the block level (containing many pages) <sup>25</sup> <sup>26</sup> . Because flash cells must be erased before being rewritten, the controller uses techniques like **out-of-place updates** and garbage collection to maintain performance – new writes go to fresh blocks, and invalidated data is cleaned later. This design means that as an SSD fills up, write operations may incur additional overhead (read–erase–modify cycles), causing **performance to degrade** from its peak “fresh-out-of-the-box” state to a steady-state level <sup>27</sup> <sup>28</sup> . In steady state, an SSD’s IOPS can be less than 50% (or even as low as 5%) of its initial peak value, depending on how aggressive the workload is and how well the controller’s firmware manages the flash <sup>28</sup> . Sophisticated controllers with large DRAM caches and effective algorithms minimize this performance drop and ensure consistency.

**Parallelism:** A key architectural advantage of SSDs is internal parallelism. To overcome the relatively slow program/erase speed of individual flash cells, SSD controllers access multiple NAND chips concurrently via independent channels <sup>17</sup> . A high-end SSD may have 8 or more flash channels, each with several chips (dies) that can operate in parallel. This allows the device to service multiple I/O requests simultaneously and aggregate the throughput of many chips <sup>29</sup> . For instance, while programming a page on one chip (which might take several hundred microseconds), the controller can issue reads or writes to other chips on different channels. By **striping data across many chips**, SSDs achieve much higher bandwidth than a single flash chip’s raw speed. In essence, an SSD internally resembles a RAID-like arrangement of flash dies managed by the controller. This internal parallelism, combined with advanced caching strategies (using DRAM or SLC-mode flash as cache), is what enables SSDs to provide very high IOPS and throughput to the host <sup>29</sup> <sup>30</sup> .

**Buffer memory:** Most SSDs include a DRAM buffer or cache integrated with the controller <sup>13</sup> . This **DDR memory** (often DDR3 or DDR4 on modern drives) is used to stage data for writes and reads, coalesce small random writes into larger sequential bursts, and cache the mapping table (FTL) for faster translation of addresses. The buffer can significantly improve performance by absorbing bursts of writes and buffering

read data, but it requires protection (e.g., supercapacitors or flash-backed buffers) to avoid data loss on power failure <sup>31</sup> <sup>32</sup>. Some lower-end consumer SSDs forego a DRAM cache to save cost, instead relying on the host memory (via NVMe Host Memory Buffer) or using a small on-controller SRAM. High-end enterprise SSDs, by contrast, have large caches and power loss protection capacitors to ensure in-flight data is safely committed to flash on power loss <sup>32</sup>.

Overall, the SSD's architecture – with its fast electronic data access, parallel multi-channel design, caching, and intelligent firmware – is geared towards maximizing throughput and minimizing latency to the storage media <sup>33</sup>. Next, we discuss how different interface technologies (SATA vs. NVMe/PCIe, and form factors like M.2 and U.2) build on this architecture to deliver varying performance levels.

## SSD Interfaces and Form Factors (SATA, PCIe/NVMe, M.2, U.2)

**Legacy SATA SSDs:** Early and mainstream SSDs often use the Serial ATA (SATA) interface, allowing them to be drop-in replacements for HDDs in PCs and servers. SATA SSDs typically come in a 2.5-inch drive form factor (or smaller mSATA cards) and use the **AHCI** protocol (Advanced Host Controller Interface) originally designed for HDDs. SATA III has a maximum theoretical transfer rate of 6 Gbps (~600 MB/s). In practice, modern SATA SSDs can approach this limit – e.g., up to ~550 MB/s sequential read/write – which is a dramatic improvement over HDD throughput but is constrained by the SATA bus <sup>9</sup>. The AHCI protocol also supports only one command queue with up to 32 outstanding commands, which limits I/O parallelism from the host side <sup>4</sup>. As a result, high-performance SATA SSDs often show random read/write IOPS in the tens of thousands (e.g. 80k–100k IOPS), an enormous gain over HDDs but still lower than what the flash media and controller could potentially achieve if not bottlenecked by the interface <sup>34</sup>.

**PCIe and NVMe SSDs:** To unleash higher performance, the industry moved to the PCI Express (PCIe) interface with the **NVMe (Non-Volatile Memory Express)** protocol specifically optimized for SSDs. NVMe SSDs connect directly to the motherboard's PCIe bus (either via an M.2 slot, a U.2 cable, or PCIe expansion card) and communicate using a streamlined command set designed for low latency and parallelism. NVMe supports **up to 64K command queues, each with up to 64K commands** – a massive increase in concurrency compared to AHCI's single queue of 32 <sup>4</sup>. NVMe is built to exploit multi-core CPUs and the internal parallelism of SSDs, allowing the host to issue many I/O requests simultaneously without the bottleneck of an intermediary host controller.

A standard NVMe SSD uses four PCIe lanes. On PCIe 3.0, this provides a raw bandwidth of about 32 Gbps (approximately 3.94 GB/s); on PCIe 4.0, up to ~64 Gbps (~7.88 GB/s) for four lanes. In real-world products, high-end PCIe 4.0 NVMe drives reach ~7,000 MB/s sequential read speeds, roughly **10–12× faster** than the ~600 MB/s cap of SATA III <sup>9</sup> <sup>35</sup>. Even PCIe 3.0 NVMe drives (around 3,000–3,500 MB/s) are about 5× the throughput of SATA SSDs. This gap is further widening with PCIe 5.0 generation drives. The latency advantage is also significant: NVMe's more efficient driver and register interface reduces software I/O overhead. For example, an AHCI command might incur ~6 microseconds of latency just in the host/controller interaction, whereas an NVMe command can do it in ~2.8 microseconds <sup>4</sup>. This is on top of the intrinsic flash access latency – but it means NVMe wastes less time per I/O on protocol overhead than SATA/AHCI.

**M.2 and U.2 form factors:** Modern SSDs come in several physical form factors. **M.2** is a small gumstick-shaped card (typically 22 mm wide, 80 mm long for the common 2280 size) that plugs directly into a motherboard slot. M.2 supports both SATA and PCIe/NVMe SSDs – some drives and slots are SATA-only,

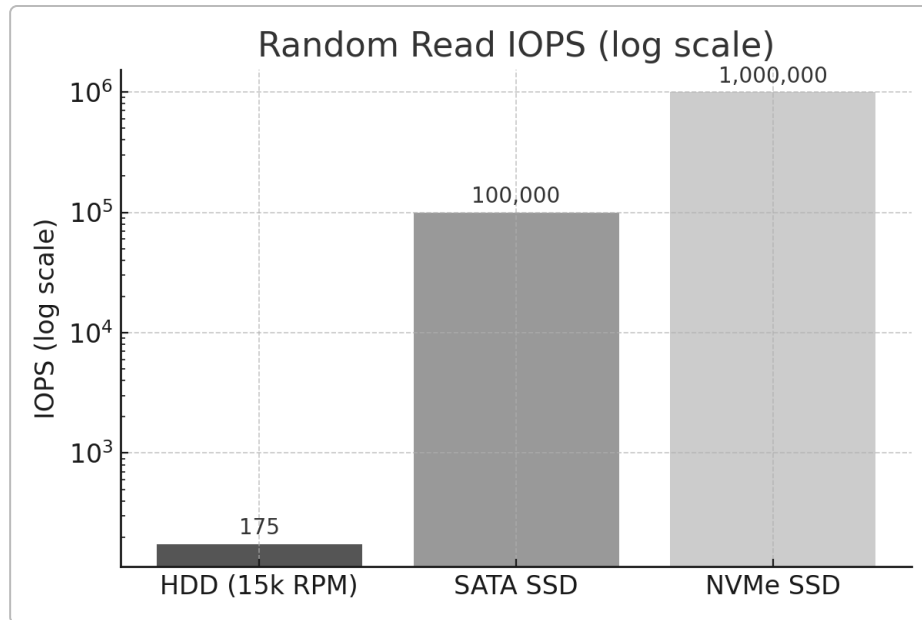
while others are NVMe. An M.2 NVMe SSD uses PCIe lanes (often 4 lanes) through the motherboard's M.2 connector, achieving the performance described above for NVMe. In contrast, an M.2 SATA SSD uses the SATA bus and offers no performance gain over a 2.5-inch SATA drive (aside from the compact form factor). **U.2** (formerly known as SFF-8639) is a 2.5-inch form factor with a different connector that typically carries four PCIe lanes and additional power lines. U.2 drives are often used in enterprise settings; they combine the small 2.5" form factor with NVMe performance and support hot-swapping via a cable backplane <sup>36</sup> <sup>37</sup> . U.2 and newer enterprise form factors like EDSFF (e.g., E3, E1.S) allow high-capacity, high-performance NVMe drives in servers with front-accessible slots, similar to how 2.5" SAS/SATA drives are used. In summary, **SATA (whether 2.5" or M.2)** tops out around 550–600 MB/s and ~100k IOPS due to interface limits, while **NVMe (PCIe)** drives in M.2, U.2, or add-in-card form can be several times faster by leveraging more bandwidth and deeper queues <sup>38</sup> <sup>15</sup> .

From an architectural perspective, the SSD's internal design (controller, flash chips, etc.) may be similar in a SATA vs. NVMe drive – indeed, many SATA SSD controllers internally support multiple flash channels just like NVMe ones. The performance differences stem largely from the **host interface**. The NVMe drives can simply push more data, with less overhead, between the host CPU and the SSD controller. The result is higher throughput and also better scaling under parallel loads (e.g., multiple threads doing I/O). The preferred interface for performance is PCIe/NVMe; as one source notes, *"the host interface is the performance bottleneck, so the preferred standard is PCIe... NVMe, using PCIe as the fabric, provides the fastest performance."* <sup>15</sup> In fact, NVMe over four lanes can theoretically achieve about **7× the transfer rate of SATA** (e.g., ~4 GB/s vs. 0.6 GB/s) <sup>39</sup> , and real devices approach these ratios.

## Performance Metrics and Comparison

**Latency and IOPS:** The elimination of mechanical delays gives SSDs an enormous advantage in latency. HDD random access time (seek + rotation) is on the order of milliseconds – a 15k RPM enterprise HDD has ~5–6 ms average rotational latency plus seek time, totaling around 8–9 ms for random I/O <sup>40</sup> . By contrast, SSDs have no seek or rotation; the latency is determined by electronic controller overhead and NAND flash cell access time. Typical read or write access latency for an SSD is well under 0.1 ms (100 μs) in many cases <sup>6</sup> <sup>38</sup> . For instance, an NVMe SSD might have a read access latency around 80–100 μs. Even SATA SSDs, despite some additional overhead, usually respond in <0.2 ms. This **two orders of magnitude** improvement in access time directly results in much higher IOPS. An HDD might only handle ~100 random 4KB IOPS (since 1 I/O takes ~10 ms) <sup>41</sup> . An SSD that can respond in 0.1–0.2 ms can handle thousands of IOPS per second from a single queue, and with queueing and parallelism, modern SSDs can reach into the hundreds of thousands of IOPS.

To quantify: a benchmark comparing a fast consumer HDD to a mainstream SSD found the HDD achieved ~208 IOPS in a 4K random write test, versus ~30,000 IOPS for the SSD <sup>8</sup> . That is about a **144× speedup**. In general, sources cite that SSDs can be *"up to three orders of magnitude faster"* than HDDs in IOPS <sup>1</sup> – meaning 1000× or more in certain random I/O scenarios. Enterprise PCIe SSDs indeed deliver 100k–1M IOPS, which is ~1000× higher than a 100–150 IOPS HDD <sup>41</sup> <sup>3</sup> . The following figure illustrates the stark contrast in random read IOPS among a 15k RPM HDD, a SATA SSD, and an NVMe SSD:



Approximate random read IOPS for a 15k RPM enterprise HDD vs. a SATA SSD vs. an NVMe SSD (log scale). The SSDs provide orders-of-magnitude higher random I/O performance than HDDs. Note the logarithmic scale – e.g., ~175 IOPS for the HDD vs. ~100k for SATA SSD vs. ~1,000k (1 million) for NVMe SSD <sup>41</sup> <sup>3</sup>.

As shown above, the NVMe drive's random IOPS can be around an order of magnitude higher than the SATA SSD, which in turn is two orders higher than the HDD. This huge gap transforms application performance: databases, virtualization, and any random-access-heavy workload benefit from SSDs' ability to service many I/Os with minimal delay <sup>42</sup> <sup>43</sup>. It is worth noting that **latency consistency** is also better with SSDs – mechanical drives have more variability (due to seeks, head contention, etc.), whereas SSD latencies are more uniform and only tend to rise when the device is saturated or when performing background flash management tasks.

**Throughput (Bandwidth):** SSDs also excel at sequential access throughput. HDD sequential transfer rates are limited by rotation speed and bit density; modern 3.5" HDDs might reach 200–250 MB/s at best (on outer tracks) and less on inner tracks <sup>10</sup>. A SATA SSD can deliver ~550 MB/s consistently across the entire drive, since flash has no moving head to reposition. This already gives a 2–3× advantage over typical HDDs in sequential reads/writes. However, when we consider NVMe SSDs on PCIe, the throughput jumps dramatically. As mentioned, PCIe 3.0 x4 NVMe drives commonly reach ~3,000 MB/s, and PCIe 4.0 drives ~7,000 MB/s – **an order of magnitude higher** than SATA SSDs <sup>9</sup> <sup>35</sup>. For example, a high-performance NVMe drive like the Samsung 970/980 Pro or WD Black can read at ~3.5 GB/s (3500 MB/s), whereas no SATA drive can exceed ~0.56 GB/s (560 MB/s). This difference is observable in tasks like large file transfers and streaming workloads: copying large media files, loading games, or sequential backups will be significantly faster on NVMe.

It should be noted that not all workloads will fully utilize the available throughput – many are limited by other factors (CPU, network, small I/O size, etc.). Thus, the real-world benefit of NVMe's extra bandwidth is most pronounced in scenarios like **large sequential I/O** or heavy concurrent I/O from multiple sources. For typical desktop use (booting OS, launching applications, moderate multitasking), a SATA SSD already feels vastly faster than an HDD, and NVMe, while technically much faster, may only shave off small fractions of a

second in user-visible latency <sup>44</sup> <sup>45</sup> . In enterprise use (databases, high-frequency trading, virtualization with many VMs), NVMe's superior throughput and low latency at scale become crucial.

**Multi-Queue and Concurrency:** Another performance aspect is how SSDs handle multiple I/O threads. NVMe's design with many queues means that it can take advantage of parallel I/O from multi-core systems without locking – multiple CPU cores can submit I/O to an NVMe drive simultaneously on different queues, greatly reducing contention and context switches <sup>46</sup> . By contrast, SATA (AHCI) has a single queue; even if multiple threads issue I/O, they funnel into one queue of 32 commands. This is sufficient for lightly concurrent workloads but becomes a limitation under highly concurrent workloads. As a result, NVMe drives scale better as you increase the I/O depth and number of CPU cores driving I/O. This is reflected in benchmarks: at high queue depths, NVMe IOPS continue to climb, whereas SATA drives often plateau. For example, an NVMe SSD can sustain hundreds of thousands of IOPS at high queue depth (QD32 or QD64), whereas a SATA SSD might reach its max ~90k–100k IOPS by QD32 and not improve beyond that <sup>47</sup> <sup>48</sup> .

**Performance vs. Capacity and Endurance:** It's worth noting that SSD performance can depend on the NAND type and drive capacity. Drives with more flash dies (higher capacity) often have more parallelism and thus higher throughput/IOPS, up to the controller's limit. Also, SLC or MLC NAND (1 or 2 bits per cell) is faster (and more enduring) than TLC or QLC (3 or 4 bits per cell). A cell storing more bits requires more precise (and thus slower) operations to read/program the correct voltage level <sup>49</sup> . Thus, **there is a trade-off between density and performance:** for instance, an SLC-based SSD (often used in enterprise cache or older generation SSDs) might have lower latency and higher write endurance, whereas a QLC SSD (used for cost-efficient storage) will have slower writes and may rely on an SLC cache to mask this. Many consumer SSDs today use TLC or QLC NAND but include an **SLC write cache** that handles incoming writes at a high speed (as pseudo-SLC), then later folds the data into the slower main area. This technique allows very high burst performance, though sustained large writes can overwhelm the cache and reveal the lower speed of TLC/QLC (sometimes dropping to a few hundred MB/s after the cache is full).

**Real-World Impact:** The architectural and interface improvements of SSDs translate to tangible benefits in real-world computing. Systems with an SSD boot in seconds versus minutes with an HDD, applications launch rapidly, and multitasking is smoother because background disk activity doesn't stall the whole system as often. In enterprise databases, moving from HDDs to SSDs (or from SATA SSDs to NVMe SSDs) can remove I/O bottlenecks, enabling the CPU and software to operate at full speed. One analysis showed that replacing arrays of 10k RPM or 15k RPM disks with SSDs yielded an **~80× increase in IOPS** in RAID configurations <sup>50</sup> <sup>51</sup> , allowing far fewer SSDs to handle the same workload as many disks. Another effect is consistency: SSDs under steady load maintain relatively consistent low response times, whereas HDDs may have long tail latencies (e.g., occasional 100+ ms seeks). This is crucial in high-performance environments where **quality of service** for storage latency matters.

Finally, the **reductions in power and cooling** are non-trivial for data centers. SSDs consume less power per I/O operation than HDDs – one study noted an HDD might use ~5× the electricity of an equivalent SSD for the same workload, and SSDs' higher density means fewer drives are needed <sup>52</sup> . This results in lower cooling requirements and can improve overall system TCO (total cost of ownership), in addition to performance per watt advantages.

## Conclusion

SSDs achieve their superior performance through a combination of architectural features and optimized interfaces. By replacing mechanical motion with electronic operations, SSDs eliminate the seek and rotation delays inherent to HDDs, cutting access latency from milliseconds to microseconds. Internally, SSD controllers use parallelism (multiple flash channels and dies) and caching to overcome NAND flash's physical program/erase limitations, thereby delivering high throughput and IOPS to the host <sup>29</sup>. The evolution of host interfaces from SATA/AHCI to PCIe/NVMe has further unlocked SSD performance: NVMe drives leverage wider bandwidth and deep queueing to fully utilize the internal speeds of flash memory, outpacing SATA SSDs by large margins <sup>3</sup> <sup>4</sup>. Empirical data and industry benchmarks consistently show drastic performance benefits of SSDs – often 100× or more in random I/O – over even the fastest HDDs <sup>1</sup> <sup>2</sup>. These gains have transformed consumer computing (enabling fast boot times, quick application launches, and snappy responsiveness) and have been instrumental in enterprise computing (improving database transaction rates, virtualization density, and big data processing).

Looking ahead, SSD architecture continues to advance. Emerging technologies such as 3D XPoint and other non-volatile memories seek to reduce latency even further by moving closer to DRAM-like speeds. The NVMe protocol and form factors will also evolve (e.g., PCIe 5.0 SSDs, NVMe over Fabrics for remote storage). Yet, the fundamental architectural theme remains: minimizing latency and maximizing parallel throughput to storage media. In that regard, SSDs will continue to be the backbone of high-performance storage. The replacement of HDDs by SSDs in ever more domains underscores the critical performance benefits delivered by solid-state storage technology.

## References (Selection)

- J. Metz and B. Lynn, *NVMe Building Blocks – Controller, Buffer Memory, Media and Form Factors*, NVM Express Blog, 2020 <sup>12</sup> <sup>17</sup>.
- Steve Larrivee, *Solid State Drive Primer – Controller Architecture*, Cactus Technologies Blog, 2015 <sup>21</sup> <sup>22</sup>.
- Kingston Technology, *Understanding SSD Technology: NVMe, SATA, M.2*, 2021 <sup>3</sup> <sup>4</sup>.
- Karl Paulsen, *Comparing IOPS for SSDs and HDDs*, TV Tech Magazine, Jul 2015 <sup>50</sup> <sup>53</sup>.
- Louwrentius Blog, *Understanding Storage Performance – IOPS and Latency*, 2018 <sup>7</sup> <sup>2</sup>.
- PhoenixNAP, *NVMe vs. SATA: Storage Comparison & Differences*, Apr 2024 <sup>38</sup> <sup>35</sup>.
- Wikipedia, *Solid-State Drive*, rev. 2025 <sup>29</sup> <sup>33</sup>. (General reference for SSD attributes)

---

<sup>1</sup> <sup>8</sup> <sup>52</sup> HDD Versus SSD: A Head to Head Comparison for Data Storage  
<https://www.zadara.com/blog/2017/01/25/hdd-versus-ssd-head-head-comparison/>

<sup>2</sup> <sup>6</sup> <sup>7</sup> <sup>10</sup> <sup>34</sup> <sup>40</sup> <sup>42</sup> <sup>43</sup> Understanding Storage Performance - IOPS and Latency  
<https://louwrentius.com/understanding-storage-performance-iops-and-latency.html>

<sup>3</sup> <sup>4</sup> <sup>9</sup> <sup>46</sup> Understanding SSD Technology: NVMe, SATA, M.2 - Kingston Technology  
<https://www.kingston.com/en/ssd/what-is-nvme-ssd-technology>

<sup>5</sup> <sup>29</sup> <sup>33</sup> Solid-state drive - Wikipedia  
[https://en.wikipedia.org/wiki/Solid-state\\_drive](https://en.wikipedia.org/wiki/Solid-state_drive)

11 14 15 23 24 25 26 28 **Data Storage: The Solid State Drive (SSD) - Lauro Rizzatti Marketing Consulting**  
<https://www.rizzatti.com/data-storage-the-solid-state-drive-ssd/>

12 13 16 17 18 19 20 30 31 32 36 37 **NVMe™ Form Factors Blog Series Part II: “NVMe Building Blocks – Controller, Buffer Memory, Media and Form Factors” - NVM Express**  
<https://nvmexpress.org/nvme-form-factors-blog-series-part-ii-nvme-building-blocks-controller-buffer-memory-media-and-form-factors/>

21 22 **Solid State Drive Primer # 9 - Controller Architecture - Controller Block Diagram**  
<https://www.cactus-tech.com/resources/blog/details/solid-state-drive-primer-9-controller-architecture-controller-block-diagram/>

27 50 51 53 **Comparing IOPS for SSDs and HDDs | TV Tech**  
<https://www.tvtechnology.com/opinions/comparing-iops-for-ssds-and-hdds>

35 38 47 48 **NVMe vs. SATA: Storage Comparison & Differences**  
<https://phoenixnap.com/kb/nvme-vs-sata>

39 **[PDF] Infographic: Transitioning to NVMe - Western Digital**  
[https://documents.westerndigital.com/content/dam/doc-library/en\\_us/assets/public/western-digital/collateral/infographic/infographic-transitioning-from-sata-nvme-future-ready-system.pdf](https://documents.westerndigital.com/content/dam/doc-library/en_us/assets/public/western-digital/collateral/infographic/infographic-transitioning-from-sata-nvme-future-ready-system.pdf)

41 **Know Your Storage Constraints: IOPS and Throughput**  
<https://www.lunavi.com/blog/know-your-storage-constraints-iops-and-throughput>

44 45 **I made this diagram to help reduce confusion regarding SSDs, NVMe, SATA and M.2 : r/pcmasterrace**  
[https://www.reddit.com/r/pcmasterrace/comments/a4kczw/i\\_made\\_this\\_diagram\\_to\\_help\\_reduce\\_confusion/](https://www.reddit.com/r/pcmasterrace/comments/a4kczw/i_made_this_diagram_to_help_reduce_confusion/)

49 **SSD Structures and Best Practice**  
<https://www.macrium.com/blog/ssd-structures-and-best-practice-1e6592368041>